

# Advanced Python Project – Carbon Emission Dataset

This notebook is prepared for the Advanced Python project on the carbon emission dataset. It includes data loading, cleaning, exploratory analysis, multivariate analysis, time series analysis, and support for answering the required analytical questions.[[file:3](#)]

## 1. Import libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

plt.style.use('seaborn-v0_8')
sns.set_palette('viridis')
pd.set_option('display.max_columns', None)
pd.set_option('display.float_format', '{:.2f}'.format)
```

## 2. Load dataset

Upload carbon\_emission\_dataset\_with\_Industry.csv to Colab before running this cell.  
[[file:3](#)]

```
file_path = '/kaggle/input/datasets/omaralramamneh/carbon-emission-dataset-with-industry/carbon_emission_dataset_with_Industry.csv'
df = pd.read_csv(file_path)

print('Dataset Shape:', df.shape)
print('\nColumns:')
print(df.columns.tolist())
print('\nFirst 5 rows:')
display(df.head())
print('\nDataset Info:')
df.info()
print('\nDescriptive Statistics:')
display(df.describe())
```

Dataset Shape: (18250, 22)

Columns:

```
['Company_ID', 'Date', 'Sector', 'Total_Energy_Consumption_kWh',
'Renewable_Energy_Consumption_kWh', 'NonRenewable_Energy_Consumption_kWh',
'Production_Output_Units', 'Supply_Chain_Transport_km',
'Supply_Chain_Transport_Mode', 'Raw_Material_Usage_kg',
'Carbon_Emission_tCO2e_TARGET', 'Energy_Cost_USD', 'Carbon_Tax_USD',
'Process_Efficiency_Percent', 'Employment_Count', 'Public_Acceptance_Index',
'Carbon_Reduction_Strategy', 'Strategy_Implementation_Cost_USD',
'Expected_Carbon_Reduction_Percent', 'Expected_Renewable_Share_Percent',
'Social_Impact_Score', 'Industry_Sectors']
```

First 5 rows:

|   | Company_ID | Date       | Sector        | Total_Energy_Consumption_kWh | R  |
|---|------------|------------|---------------|------------------------------|----|
| 0 | C001       | 2024-01-01 | Manufacturing | 53751.61                     | 12 |
| 1 | C001       | 2024-01-02 | Manufacturing | 170872.89                    | 76 |
| 2 | C001       | 2024-01-03 | Manufacturing | 142777.96                    | 75 |
| 3 | C001       | 2024-01-04 | Manufacturing | 152532.15                    | 20 |
| 4 | C001       | 2024-01-05 | Manufacturing | 81647.43                     | 46 |

Dataset Info:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 18250 entries, 0 to 18249

Data columns (total 22 columns):

| #   | Column                              | Non-Null Count | Dtype   |
|-----|-------------------------------------|----------------|---------|
| --- | -----                               | -----          | -----   |
| 0   | Company_ID                          | 18250 non-null | object  |
| 1   | Date                                | 18250 non-null | object  |
| 2   | Sector                              | 18250 non-null | object  |
| 3   | Total_Energy_Consumption_kWh        | 18250 non-null | float64 |
| 4   | Renewable_Energy_Consumption_kWh    | 18250 non-null | float64 |
| 5   | NonRenewable_Energy_Consumption_kWh | 18250 non-null | float64 |
| 6   | Production_Output_Units             | 18250 non-null | float64 |
| 7   | Supply_Chain_Transport_km           | 18250 non-null | float64 |
| 8   | Supply_Chain_Transport_Mode         | 18250 non-null | object  |
| 9   | Raw_Material_Usage_kg               | 18250 non-null | float64 |
| 10  | Carbon_Emission_tCO2e_TARGET        | 18250 non-null | float64 |
| 11  | Energy_Cost_USD                     | 18250 non-null | float64 |
| 12  | Carbon_Tax_USD                      | 18250 non-null | float64 |
| 13  | Process_Efficiency_Percent          | 18250 non-null | float64 |
| 14  | Employment_Count                    | 18250 non-null | int64   |
| 15  | Public_Acceptance_Index             | 18250 non-null | float64 |
| 16  | Carbon_Reduction_Strategy           | 18250 non-null | object  |
| 17  | Strategy_Implementation_Cost_USD    | 18250 non-null | float64 |
| 18  | Expected_Carbon_Reduction_Percent   | 18250 non-null | float64 |
| 19  | Expected_Renewable_Share_Percent    | 18250 non-null | float64 |
| 20  | Social_Impact_Score                 | 18250 non-null | float64 |

21 Industry\_Sectors 18250 non-null object  
dtypes: float64(15), int64(1), object(6)  
memory usage: 3.1+ MB

Descriptive Statistics:

|       | Total_Energy_Consumption_kWh | Renewable_Energy_Consumption_kV |
|-------|------------------------------|---------------------------------|
| count | 18250.00                     | 18250.00                        |
| mean  | 124776.99                    | 43759.10                        |
| std   | 43106.62                     | 24291.67                        |
| min   | 50008.73                     | 5041.20                         |
| 25%   | 87326.36                     | 24455.50                        |
| 50%   | 124728.53                    | 38888.00                        |
| 75%   | 162042.96                    | 59545.03                        |
| max   | 199988.91                    | 119005.26                       |

### 3. Standardize column names

```
df.columns = [col.strip() for col in df.columns]
```

```
rename_map = {  
    'CompanyID': 'Company_ID',  
    'Date': 'Date',  
    'Sector': 'Sector',  
    'TotalEnergyConsumptionkWh': 'Total_Energy_Consumption_kWh',  
    'RenewableEnergyConsumptionkWh': 'Renewable_Energy_Consumption_kWh',  
    'NonRenewableEnergyConsumptionkWh':  
        'NonRenewable_Energy_Consumption_kWh',  
    'ProductionOutputUnits': 'Production_Output_Units',  
    'SupplyChainTransportkm': 'Supply_Chain_Transport_km',  
    'SupplyChainTransportMode': 'Supply_Chain_Transport_Mode',  
    'RawMaterialUsagekg': 'Raw_Material_Usage_kg',  
    'CarbonEmissiontCO2eTARGET': 'Carbon_Emission_tCO2e_TARGET',  
    'EnergyCostUSD': 'Energy_Cost_USD',  
    'CarbonTaxUSD': 'Carbon_Tax_USD',  
    'ProcessEfficiencyPercent': 'Process_Efficiency_Percent',  
    'EmploymentCount': 'Employment_Count',  
    'PublicAcceptanceIndex': 'Public_Acceptance_Index',  
    'CarbonReductionStrategy': 'Carbon_Reduction_Strategy',  
    'StrategyImplementationCostUSD': 'Strategy_Implementation_Cost_USD',  
    'ExpectedCarbonReductionPercent': 'Expected_Carbon_Reduction_Percent',  
    'ExpectedRenewableSharePercent': 'Expected_Renewable_Share_Percent',  
    'SocialImpactScore': 'Social_Impact_Score',  
    'IndustrySectors': 'Industry_Sectors'  
}
```

```
df.rename(columns=rename_map, inplace=True)
print(df.columns.tolist())

['Company_ID', 'Date', 'Sector', 'Total_Energy_Consumption_kWh',
'Renewable_Energy_Consumption_kWh', 'NonRenewable_Energy_Consumption_kWh',
'Production_Output_Units', 'Supply_Chain_Transport_km',
'Supply_Chain_Transport_Mode', 'Raw_Material_Usage_kg',
'Carbon_Emission_tCO2e_TARGET', 'Energy_Cost_USD', 'Carbon_Tax_USD',
'Process_Efficiency_Percent', 'Employment_Count', 'Public_Acceptance_Index',
'Carbon_Reduction_Strategy', 'Strategy_Implementation_Cost_USD',
'Expected_Carbon_Reduction_Percent', 'Expected_Renewable_Share_Percent',
'Social_Impact_Score', 'Industry_Sectors']
```

## 4. Data cleaning

```
print('Missing Values:')
display(df.isnull().sum())

print('Duplicate Rows:', df.duplicated().sum())
df = df.drop_duplicates()

if 'Date' in df.columns:
    df['Date'] = pd.to_datetime(df['Date'], errors='coerce')

numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
category_cols = df.select_dtypes(include=['object']).columns.tolist()

for col in numeric_cols:
    df[col] = df[col].fillna(df[col].median())

for col in category_cols:
    if df[col].mode().empty:
        df[col] = df[col].fillna('Unknown')
    else:
        df[col] = df[col].fillna(df[col].mode()[0])

print('\nData types after cleaning:')
display(df.dtypes)
```

Missing Values:

|                                     |   |
|-------------------------------------|---|
| Company_ID                          | 0 |
| Date                                | 0 |
| Sector                              | 0 |
| Total_Energy_Consumption_kWh        | 0 |
| Renewable_Energy_Consumption_kWh    | 0 |
| NonRenewable_Energy_Consumption_kWh | 0 |
| Production_Output_Units             | 0 |
| Supply_Chain_Transport_km           | 0 |
| Supply_Chain_Transport_Mode         | 0 |
| Raw_Material_Usage_kg               | 0 |
| Carbon_Emission_tCO2e_TARGET        | 0 |

```

Energy_Cost_USD          0
Carbon_Tax_USD           0
Process_Efficiency_Percent 0
Employment_Count         0
Public_Acceptance_Index  0
Carbon_Reduction_Strategy 0
Strategy_Implementation_Cost_USD 0
Expected_Carbon_Reduction_Percent 0
Expected_Renewable_Share_Percent 0
Social_Impact_Score      0
Industry_Sectors         0
dtype: int64

```

Duplicate Rows: 0

Data types after cleaning:

```

Company_ID              object
Date                   datetime64[ns]
Sector                 object
Total_Energy_Consumption_kWh float64
Renewable_Energy_Consumption_kWh float64
NonRenewable_Energy_Consumption_kWh float64
Production_Output_Units float64
Supply_Chain_Transport_km float64
Supply_Chain_Transport_Mode object
Raw_Material_Usage_kg   float64
Carbon_Emission_tCO2e_TARGET float64
Energy_Cost_USD        float64
Carbon_Tax_USD         float64
Process_Efficiency_Percent float64
Employment_Count       int64
Public_Acceptance_Index float64
Carbon_Reduction_Strategy object
Strategy_Implementation_Cost_USD float64
Expected_Carbon_Reduction_Percent float64
Expected_Renewable_Share_Percent float64
Social_Impact_Score    float64
Industry_Sectors       object
dtype: object

```

## 5. Basic statistics

```

selected_cols = [
    'Total_Energy_Consumption_kWh',
    'Renewable_Energy_Consumption_kWh',
    'NonRenewable_Energy_Consumption_kWh',
    'Production_Output_Units',
    'Supply_Chain_Transport_km'
]

```

```

selected_cols = [c for c in selected_cols if c in df.columns]

```

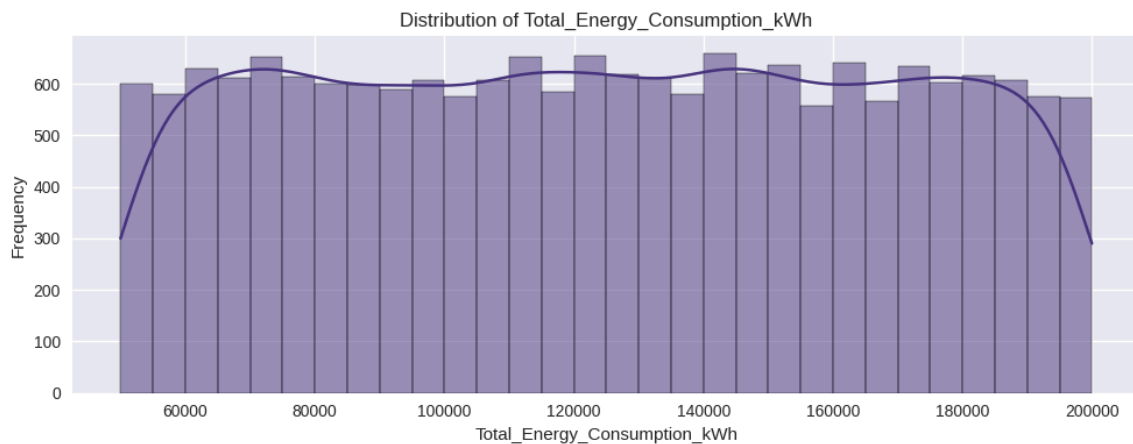
```
basic_stats = df[selected_cols].agg(['mean', 'median', 'max', 'min']).T
display(basic_stats)
```

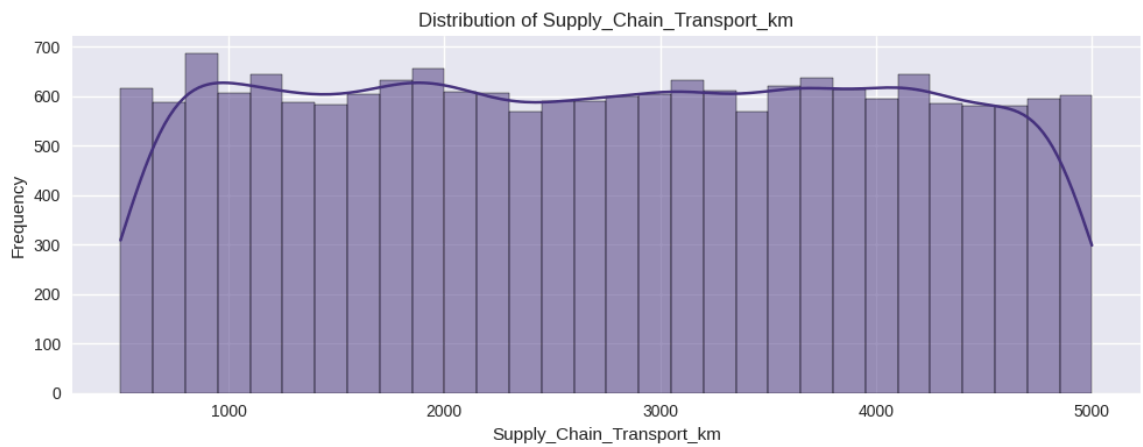
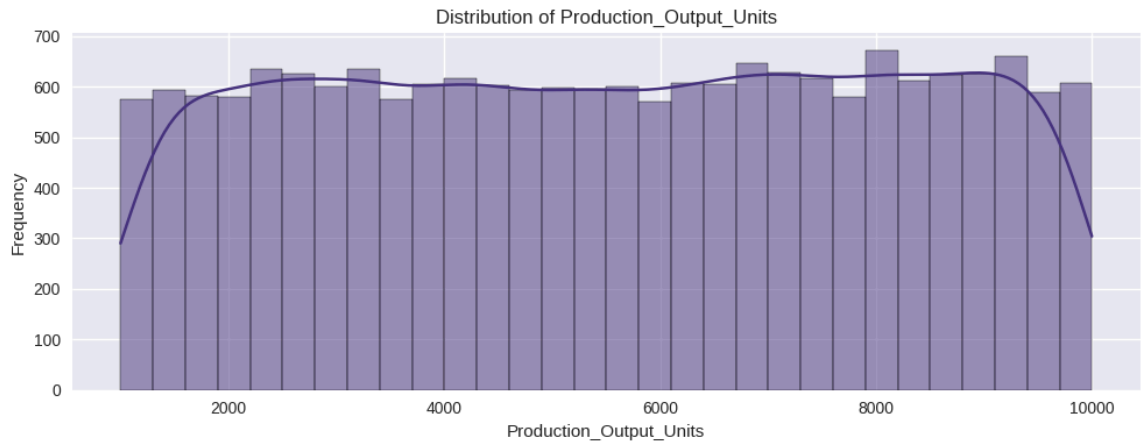
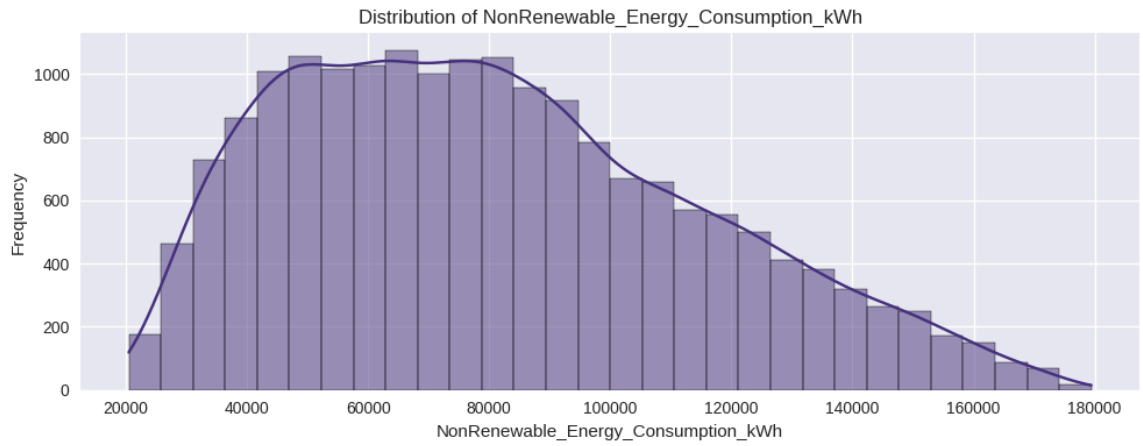
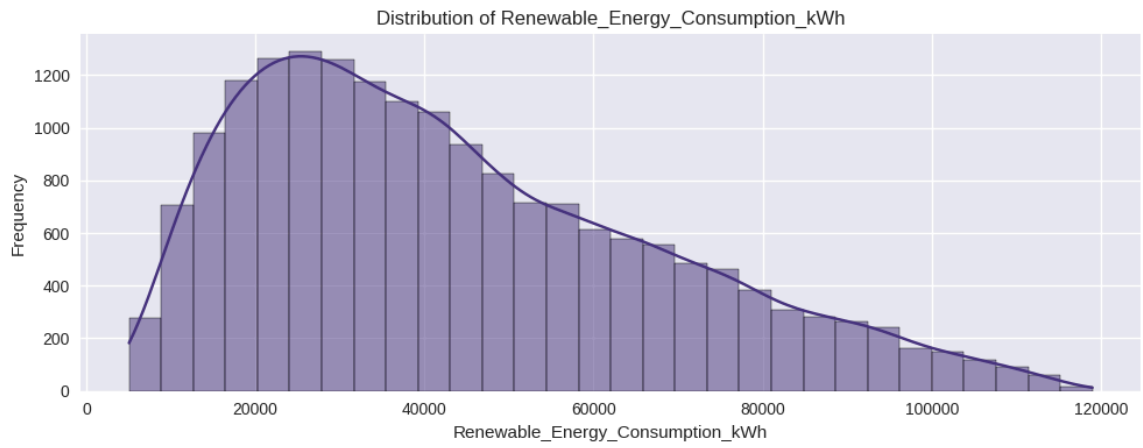
|                                            | mean      | median    | max       |
|--------------------------------------------|-----------|-----------|-----------|
| <b>Total_Energy_Consumption_kWh</b>        | 124776.99 | 124728.53 | 199988.91 |
| <b>Renewable_Energy_Consumption_kWh</b>    | 43759.10  | 38888.00  | 119005.26 |
| <b>NonRenewable_Energy_Consumption_kWh</b> | 81017.89  | 77007.82  | 179431.02 |
| <b>Production_Output_Units</b>             | 5538.96   | 5556.40   | 9999.78   |
| <b>Supply_Chain_Transport_km</b>           | 2734.81   | 2735.02   | 4999.89   |

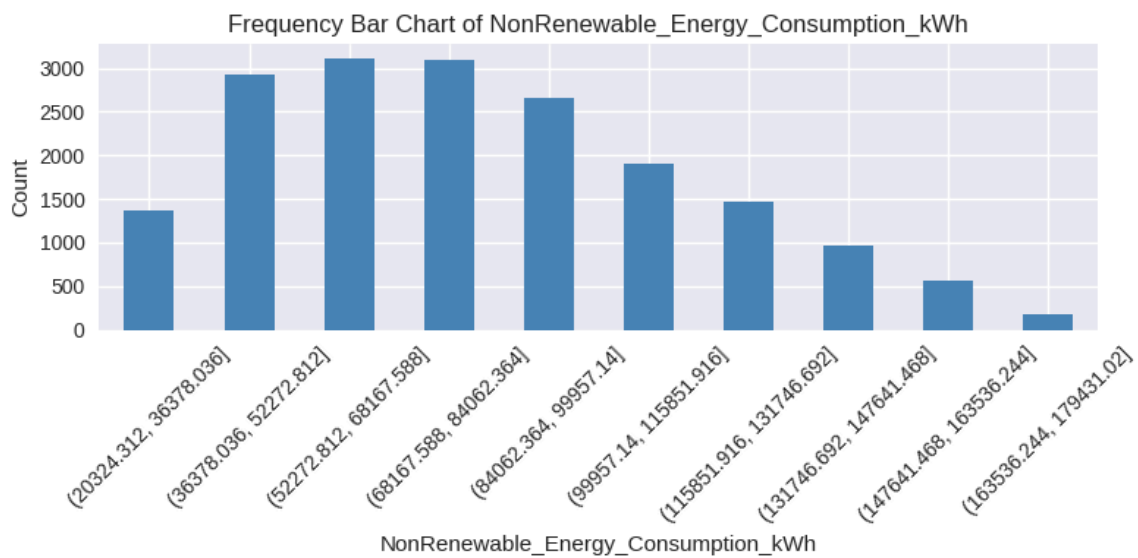
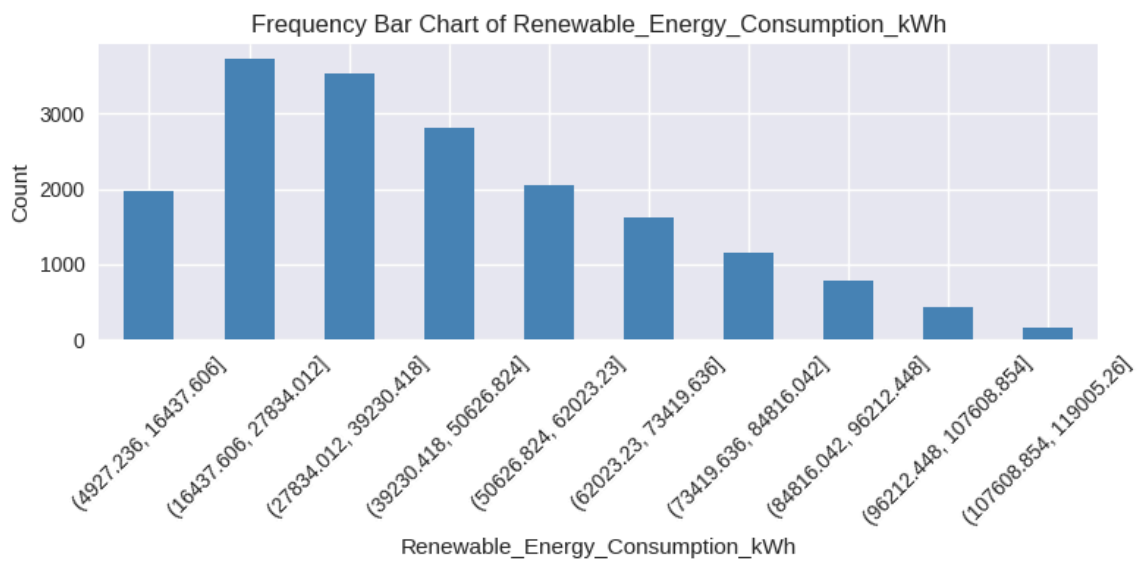
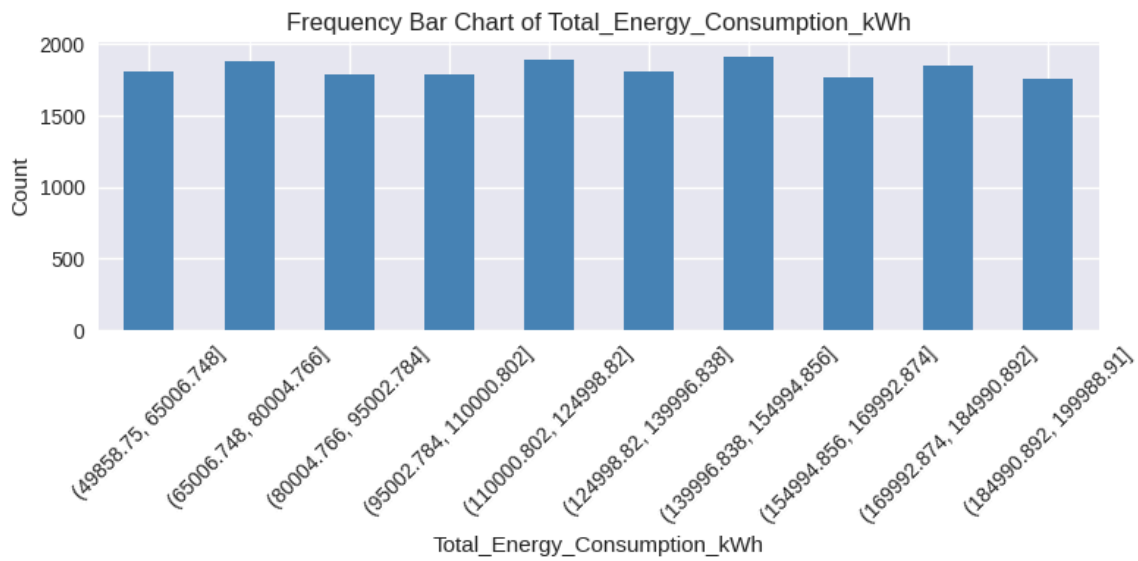
## 6. Univariate analysis

```
for col in selected_cols:
    plt.figure(figsize=(10, 4))
    sns.histplot(df[col], kde=True, bins=30)
    plt.title(f'Distribution of {col}')
    plt.xlabel(col)
    plt.ylabel('Frequency')
    plt.tight_layout()
    plt.show()
```

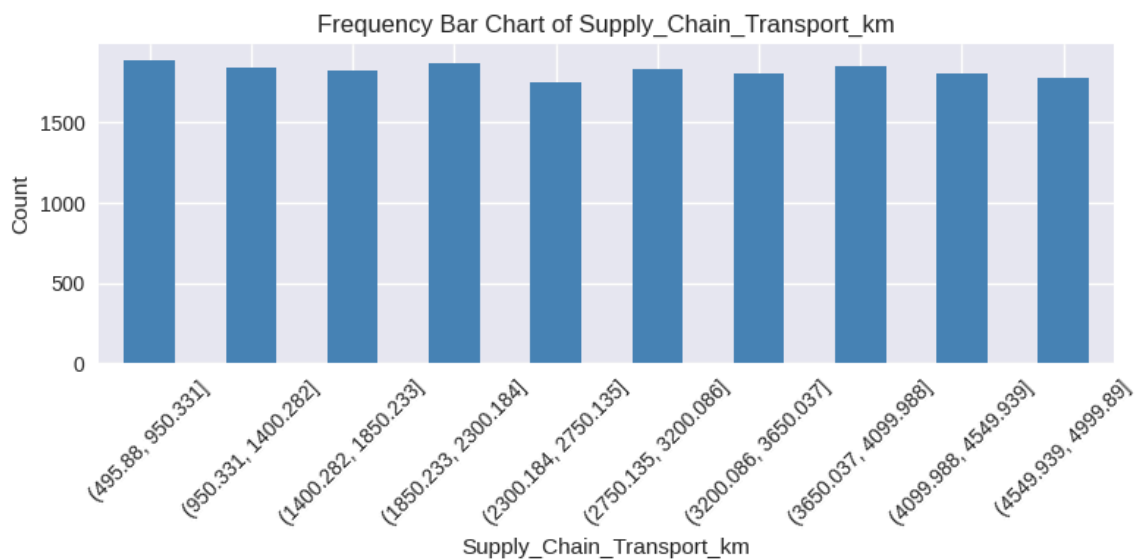
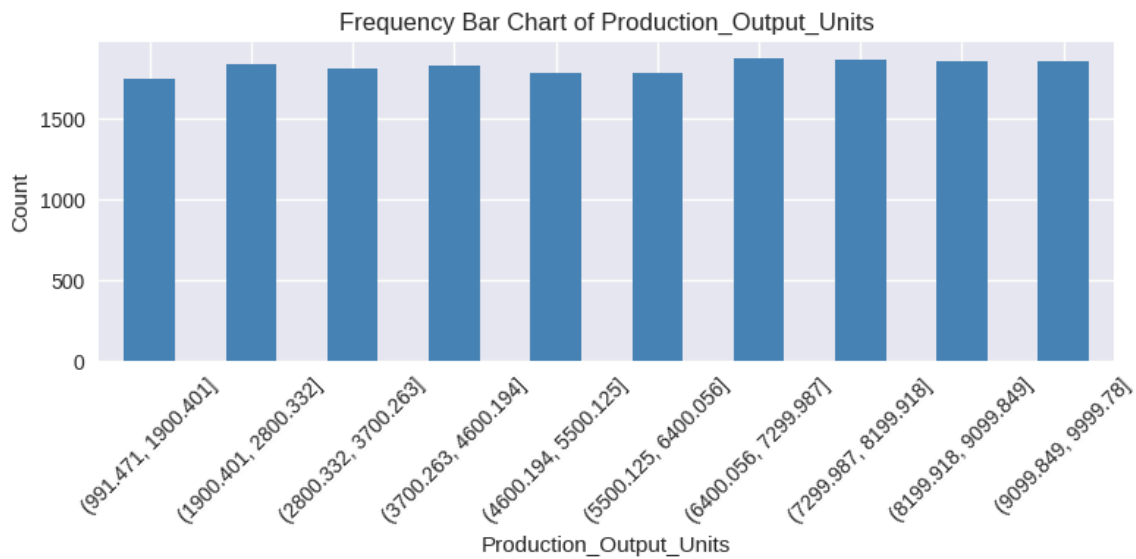
```
for col in selected_cols:
    plt.figure(figsize=(8, 4))
    binned = pd.cut(df[col], bins=10)
    binned.value_counts().sort_index().plot(kind='bar', color='steelblue')
    plt.title(f'Frequency Bar Chart of {col}')
    plt.xlabel(col)
    plt.ylabel('Count')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```











## 7. Line plot of energies over time

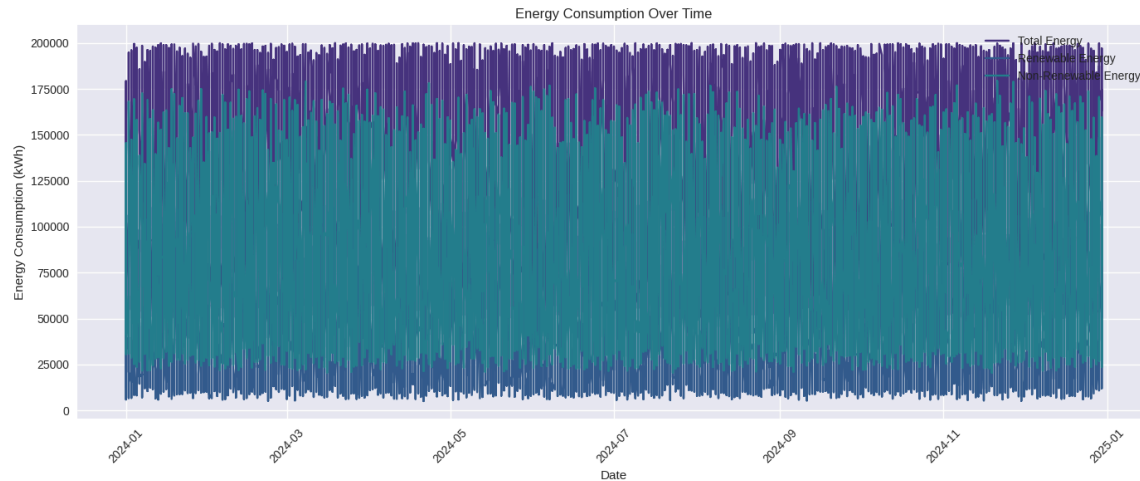
```
target_col = 'Carbon_Emission_tCO2e_TARGET'
temp_candidates = ['Temperature', 'Temperature_C', 'Avg_Temperature_C']
temp_col = next((c for c in temp_candidates if c in df.columns), None)

if 'Date' in df.columns:
    df_sorted = df.sort_values('Date')
    plt.figure(figsize=(14, 6))
    if 'Total_Energy_Consumption_kWh' in df.columns:
        plt.plot(df_sorted['Date'],
                 df_sorted['Total_Energy_Consumption_kWh'], label='Total Energy')
    if 'Renewable_Energy_Consumption_kWh' in df.columns:
        plt.plot(df_sorted['Date'],
                 df_sorted['Renewable_Energy_Consumption_kWh'], label='Renewable
                 Energy')
    if 'NonRenewable_Energy_Consumption_kWh' in df.columns:
```

```

plt.plot(df_sorted['Date'],
         df_sorted['NonRenewable_Energy_Consumption_kWh'], label='Non-
         Renewable Energy')
plt.title('Energy Consumption Over Time')
plt.xlabel('Date')
plt.ylabel('Energy Consumption (kWh)')
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```



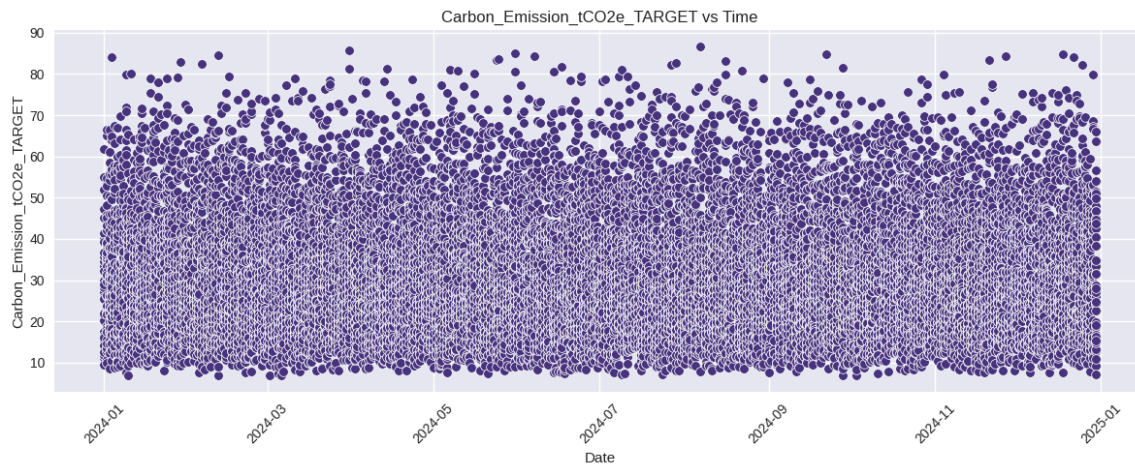
## 8. Scatter plots

```

if temp_col is not None and target_col in df.columns:
    plt.figure(figsize=(8, 5))
    sns.scatterplot(data=df, x=temp_col, y=target_col)
    plt.title(f'{target_col} vs {temp_col}')
    plt.tight_layout()
    plt.show()

if 'Date' in df.columns and target_col in df.columns:
    plt.figure(figsize=(12, 5))
    sns.scatterplot(data=df_sorted, x='Date', y=target_col)
    plt.title(f'{target_col} vs Time')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()

```



## 9. Bivariate and multivariate analysis

```
if target_col in df.columns and 'Total_Energy_Consumption_kWh' in df.columns:
    plt.figure(figsize=(8, 5))
    sns.scatterplot(data=df, x='Total_Energy_Consumption_kWh', y=target_col)
    plt.title('Carbon Emissions vs Total Energy Consumption')
    plt.tight_layout()
    plt.show()
```

```
if 'Renewable_Energy_Consumption_kWh' in df.columns and
    'NonRenewable_Energy_Consumption_kWh' in df.columns:
    plt.figure(figsize=(8, 5))
    sns.scatterplot(data=df, x='Renewable_Energy_Consumption_kWh',
                    y='NonRenewable_Energy_Consumption_kWh')
    plt.title('Renewable vs Non-Renewable Energy Consumption')
    plt.tight_layout()
    plt.show()
```

```
num_cols = df.select_dtypes(include=[np.number]).columns
corr_matrix = df[num_cols].corr()
```

```
plt.figure(figsize=(14, 10))
sns.heatmap(corr_matrix, cmap='coolwarm', annot=False)
plt.title('Correlation Heatmap')
plt.tight_layout()
plt.show()
```

```
if 'Industry_Sectors' in df.columns and target_col in df.columns:
    plt.figure(figsize=(14, 6))
    sns.boxplot(data=df, x='Industry_Sectors', y=target_col)
    plt.title('Carbon Emissions by Industry Sectors')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```

```
pairplot_cols = [
    'Carbon_Emission_tCO2e_TARGET',
```

```

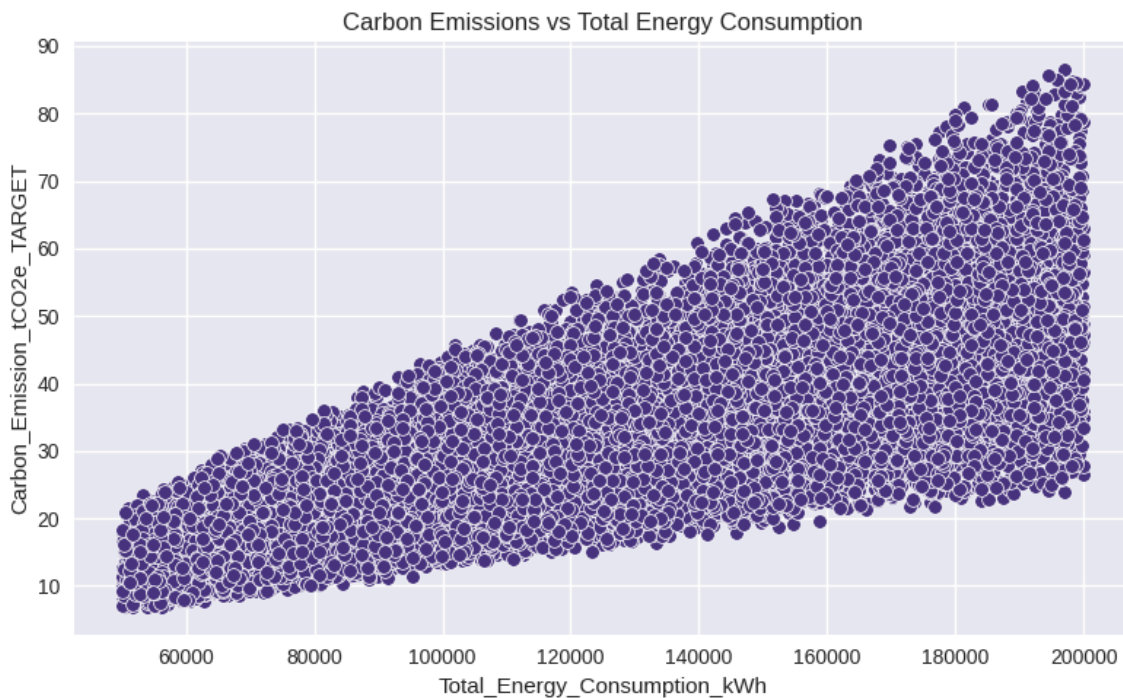
'Total_Energy_Consumption_kWh',
'Renewable_Energy_Consumption_kWh',
'NonRenewable_Energy_Consumption_kWh',
'Production_Output_Units'
]
pairplot_cols = [c for c in pairplot_cols if c in df.columns]

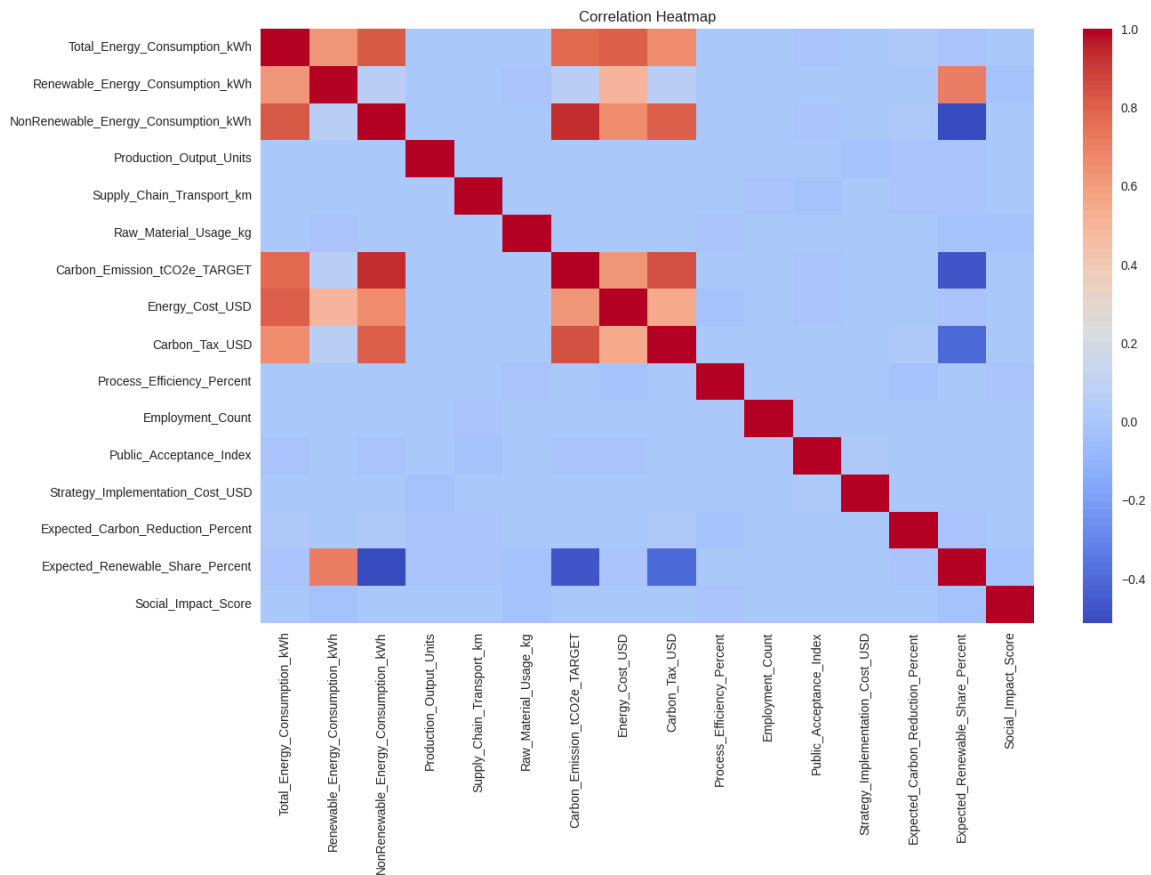
if len(pairplot_cols) >= 2:
    sns.pairplot(df[pairplot_cols], diag_kind='kde')
    plt.show()

if 'Industry_Sectors' in df.columns and target_col in df.columns:
    emissions_by_sector = df.groupby('Industry_Sectors')
    [target_col].mean().sort_values(ascending=False)
    display(emissions_by_sector)

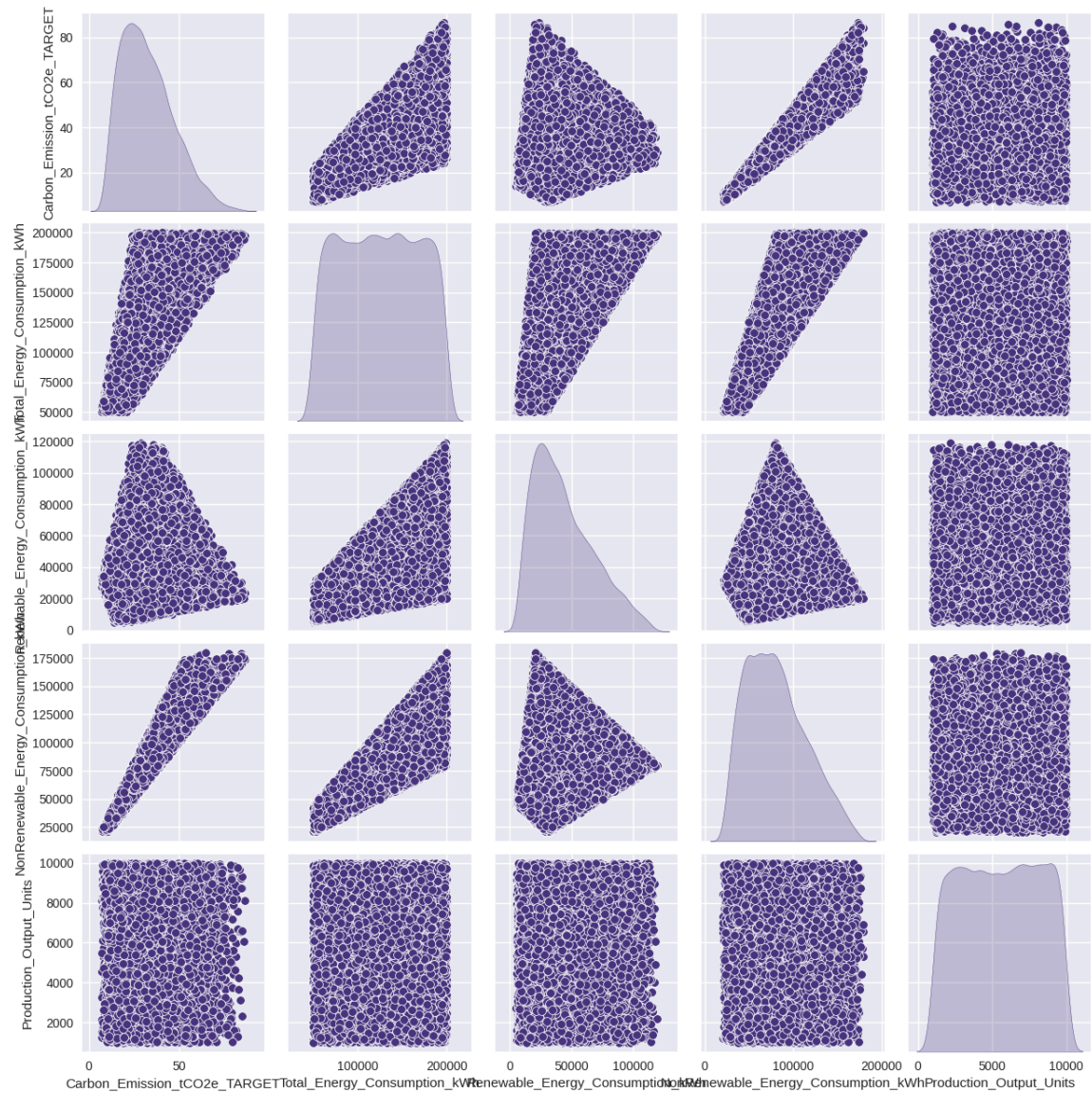
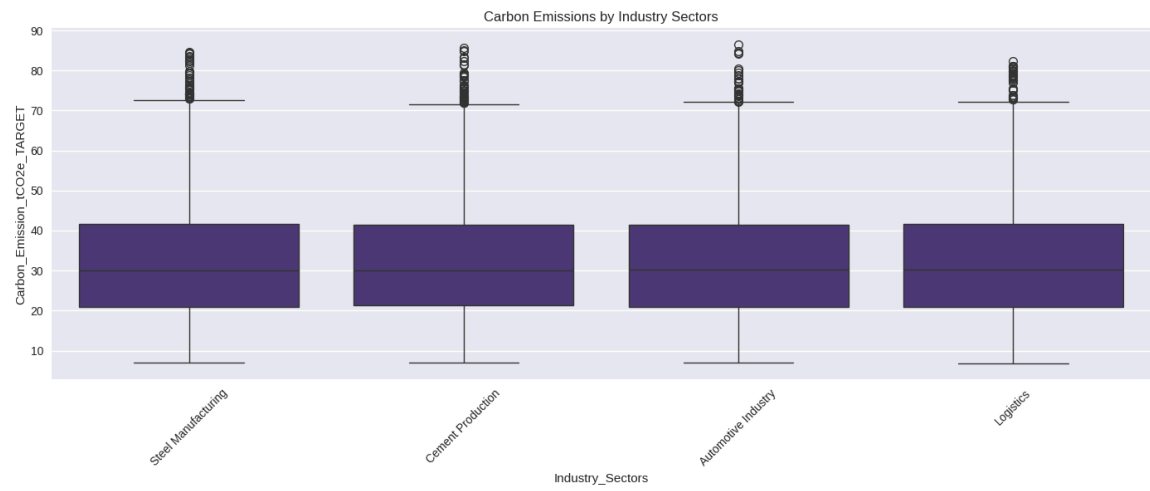
plt.figure(figsize=(14, 6))
emissions_by_sector.plot(kind='bar', color='teal')
plt.title('Average Carbon Emissions by Industry Sector')
plt.xlabel('Industry Sector')
plt.ylabel('Average Carbon Emission')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```





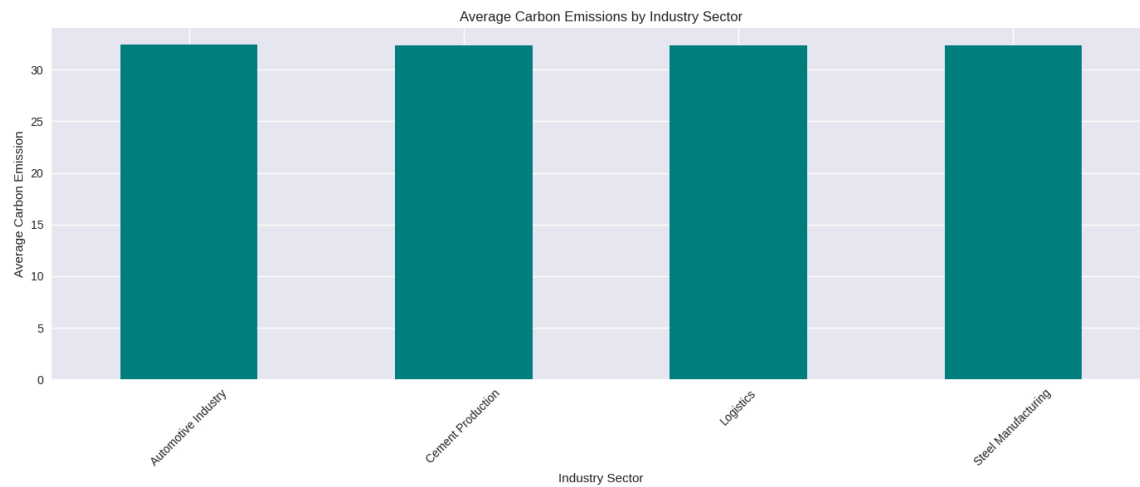




```

Industry_Sectors
Automotive Industry    32.40
Cement Production      32.36
Logistics              32.35
Steel Manufacturing    32.34
Name: Carbon_Emission_tCO2e_TARGET, dtype: float64

```



## 10. Time series analysis

```

if 'Date' in df.columns and target_col in df.columns:
    df_ts = df.sort_values('Date').set_index('Date')

    plt.figure(figsize=(14, 5))
    df_ts[target_col].plot()
    plt.title('Carbon Emissions Over Time')
    plt.ylabel('Carbon Emission (tCO2e)')
    plt.tight_layout()
    plt.show()

    if temp_col is not None:
        plt.figure(figsize=(14, 5))
        df_ts[temp_col].plot(color='orange')
        plt.title(f'{temp_col} Trend Over Time')
        plt.tight_layout()
        plt.show()

    df_ts['Rolling_Mean_30'] = df_ts[target_col].rolling(window=30).mean()
    df_ts['Rolling_Var_30'] = df_ts[target_col].rolling(window=30).var()

    plt.figure(figsize=(14, 5))
    plt.plot(df_ts.index, df_ts[target_col], label='Original', alpha=0.5)
    plt.plot(df_ts.index, df_ts['Rolling_Mean_30'], label='30-Day Moving
        Average', color='red')
    plt.title('Carbon Emissions with 30-Day Moving Average')
    plt.legend()
    plt.tight_layout()
    plt.show()

    plt.figure(figsize=(14, 5))
    plt.plot(df_ts.index, df_ts['Rolling_Var_30'], label='30-Day Rolling
        Variance', color='green')
    plt.title('30-Day Rolling Variance of Carbon Emissions')
    plt.legend()
    plt.tight_layout()

```

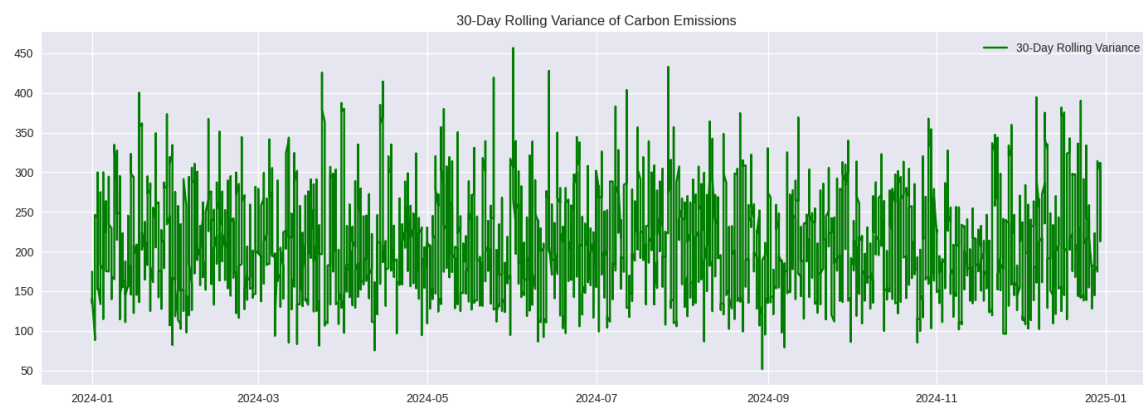
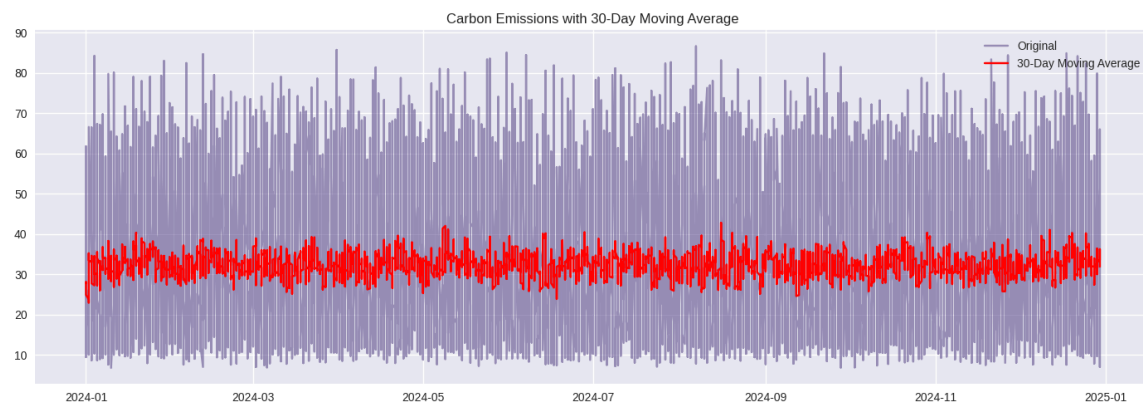
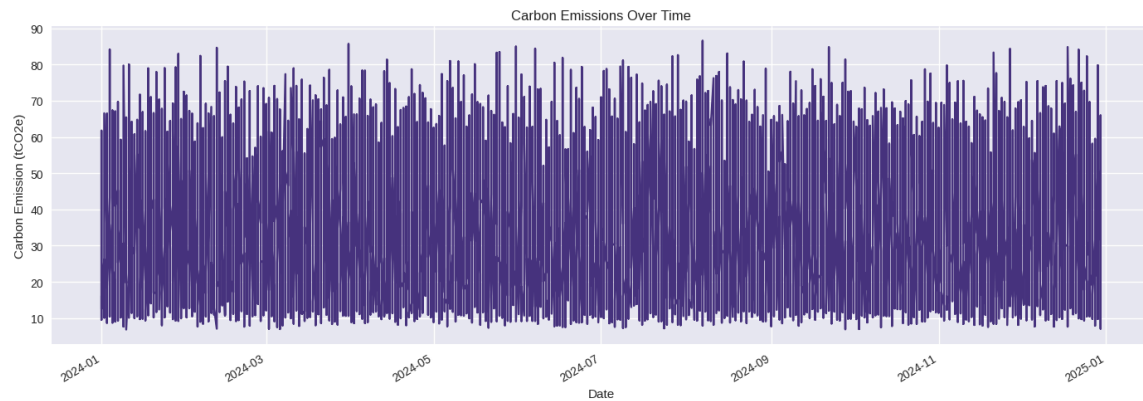
```
plt.show()
```

```
try:
```

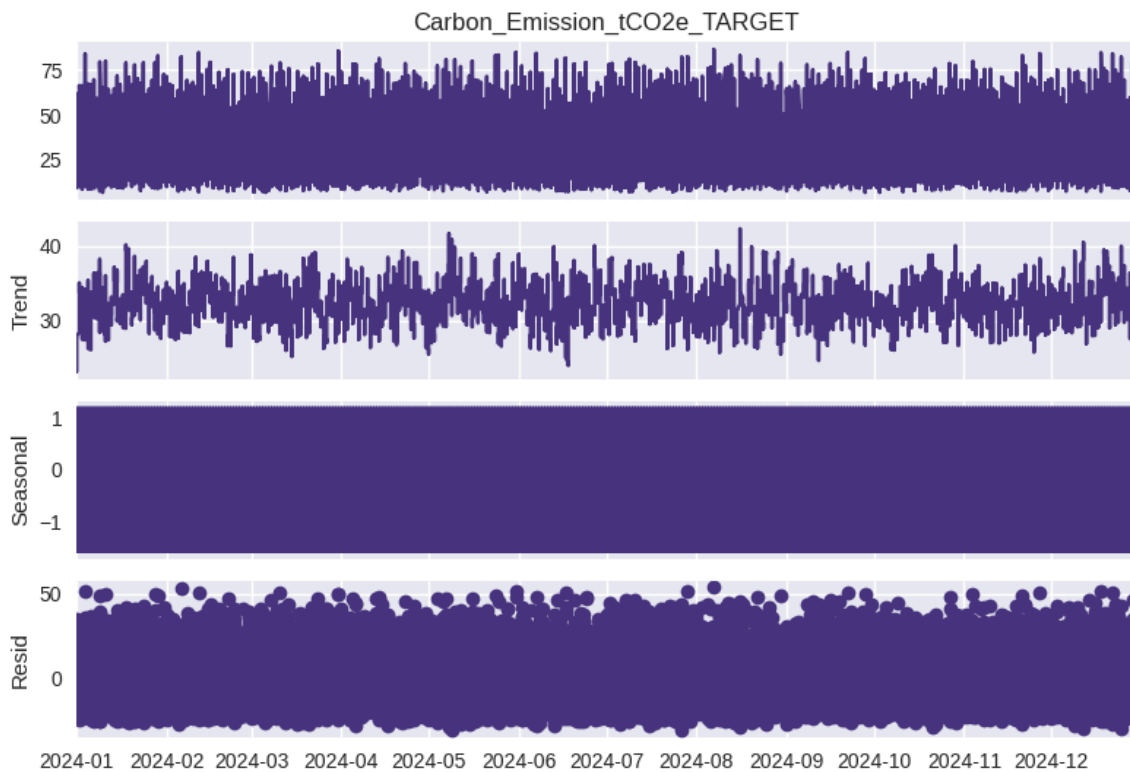
```
    from statsmodels.tsa.seasonal import seasonal_decompose
    decompose_result = seasonal_decompose(df_ts[target_col].dropna(),
        model='additive', period=30)
    decompose_result.plot()
    plt.tight_layout()
    plt.show()
```

```
except Exception as e:
```

```
    print('Seasonal decomposition could not be completed:', e)
```







## 11. Support for Q1 to Q5

```
print('Q1: Which emission sources have the highest variability?')
variability = df[selected_cols].std().sort_values(ascending=False)
display(variability)

print('Q2: Are emissions normally distributed?')
if target_col in df.columns:
    plt.figure(figsize=(8, 4))
    sns.histplot(df[target_col], kde=True, bins=30)
    plt.title('Distribution of Carbon Emissions')
    plt.tight_layout()
    plt.show()

print('Q3: Which variables are strongly correlated with carbon emissions?')
if target_col in corr_matrix.columns:
    display(corr_matrix[target_col].sort_values(ascending=False))

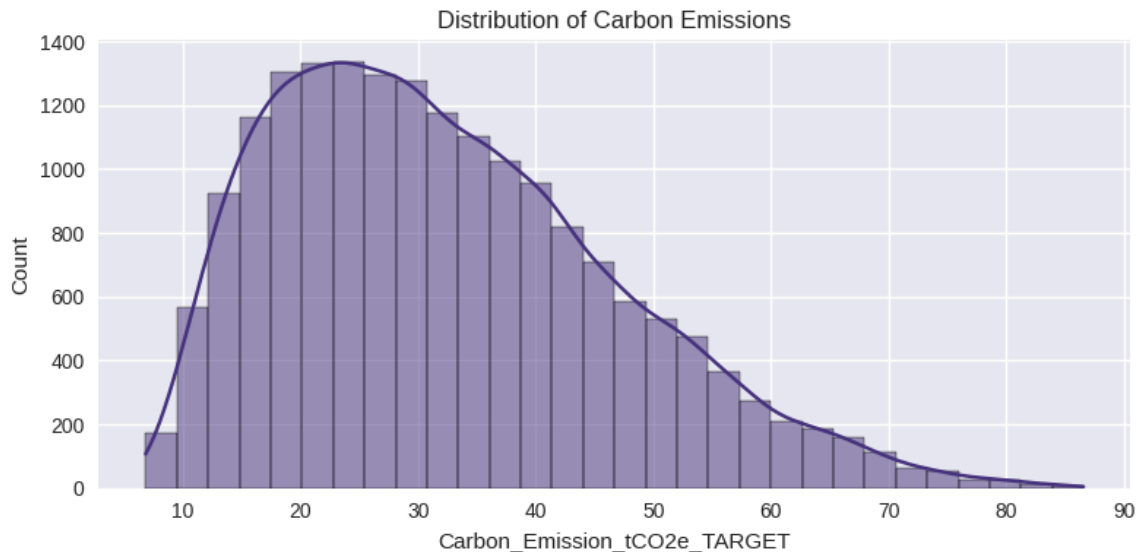
print('Q4: Is there a trend in emissions?')
print('Use the time series plot and the rolling mean plot to decide whether
      the trend is increasing, decreasing, or stable.')

print('Q5: Do seasonal patterns exist?')
print('Use the seasonal decomposition plot and repeated patterns in the time
      series to answer this question.')
```

Q1: Which emission sources have the highest variability?

|                                     |          |
|-------------------------------------|----------|
| Total_Energy_Consumption_kWh        | 43106.62 |
| NonRenewable_Energy_Consumption_kWh | 33957.85 |
| Renewable_Energy_Consumption_kWh    | 24291.67 |
| Production_Output_Units             | 2597.58  |
| Supply_Chain_Transport_km           | 1300.15  |
| dtype: float64                      |          |

Q2: Are emissions normally distributed?



Q3: Which variables are strongly correlated with carbon emissions?

|                                                    |       |
|----------------------------------------------------|-------|
| Carbon_Emission_tCO2e_TARGET                       | 1.00  |
| NonRenewable_Energy_Consumption_kWh                | 0.94  |
| Carbon_Tax_USD                                     | 0.86  |
| Total_Energy_Consumption_kWh                       | 0.78  |
| Energy_Cost_USD                                    | 0.63  |
| Renewable_Energy_Consumption_kWh                   | 0.07  |
| Expected_Carbon_Reduction_Percent                  | 0.01  |
| Raw_Material_Usage_kg                              | 0.01  |
| Social_Impact_Score                                | 0.01  |
| Production_Output_Units                            | 0.01  |
| Employment_Count                                   | 0.00  |
| Supply_Chain_Transport_km                          | 0.00  |
| Strategy_Implementation_Cost_USD                   | 0.00  |
| Process_Efficiency_Percent                         | -0.00 |
| Public_Acceptance_Index                            | -0.01 |
| Expected_Renewable_Share_Percent                   | -0.48 |
| Name: Carbon_Emission_tCO2e_TARGET, dtype: float64 |       |

Q4: Is there a trend in emissions?

Use the time series plot and the rolling mean plot to decide whether the trend is increasing, decreasing, or stable.

Q5: Do seasonal patterns exist?

Use the seasonal decomposition plot and repeated patterns in the time series to answer this question.

# 12. Final preview

```
display(df.head())
```

|   | Company_ID | Date       | Sector        | Total_Energy_Consumption_kWh | R  |
|---|------------|------------|---------------|------------------------------|----|
| 0 | C001       | 2024-01-01 | Manufacturing | 53751.61                     | 12 |
| 1 | C001       | 2024-01-02 | Manufacturing | 170872.89                    | 76 |
| 2 | C001       | 2024-01-03 | Manufacturing | 142777.96                    | 75 |
| 3 | C001       | 2024-01-04 | Manufacturing | 152532.15                    | 20 |
| 4 | C001       | 2024-01-05 | Manufacturing | 81647.43                     | 46 |